

Chapter 3 – Functions with Parameters

SLIDE 1 — What Are Parameters?

Definition 🧠

A parameter is a variable used to receive a value inside a function.

Parameters allow a function to work with different inputs.

Without parameters, a function usually performs the same fixed task. With parameters, the same function can work with different values.

Analogy 🍔

Think of a food-ordering function:

- The function is the order system.
- The parameter is the food item.
- Different inputs produce different orders.

Same function + different parameters = different results

Small Example 💻

```
None
void greet(char name[]) {
    printf("Hello %s", name);
}
```

Here, `name` is the parameter.

SLIDE 2 — Why Are Parameters Used?

Why do we use parameters? 🤔

Parameters are used to:

1. Pass information into a function.
2. Reuse the same function with different values.

3. Avoid writing separate functions for similar tasks.
4. Make programs flexible and useful.
5. Improve code organization.
6. Reduce repeated code.

Important Statement ★

Parameters allow a function to receive data from outside.

Base → Professional

Base Level:

None

```
void square(int n) {  
    printf("%d", n * n);  
}
```

Professional Level:

None

```
int calculateTotal(int price, int quantity) {  
    return price * quantity;  
}
```

The professional function accepts two values and returns a result.

Parameter vs Argument

Term	Meaning
Parameter	Variable written in the function definition
Argument	Actual value passed during the function call

Example:

None

```
void add(int a, int b)
```

- a and b → Parameters

None

```
add(5, 3);
```

- 5 and 3 → Arguments

SLIDE 3 — Types of Parameters

1. One Parameter

A function receives one value.

None

```
void display(int n) {  
    printf("%d", n);  
}
```

2. Multiple Parameters

A function receives more than one value.

None

```
void add(int a, int b) {  
    printf("%d", a + b);  
}
```

3. Parameters with Return Value

A function receives values and sends back a result.

None

```
int multiply(int a, int b) {  
    return a * b;  
}
```

Important Statement

The number, order, and type of arguments should match the parameters.

Example

None

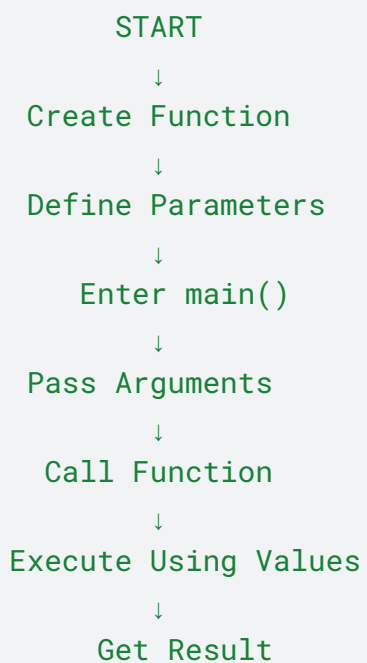
```
add(5, 3);
```

- First argument **5** goes to **a**.
- Second argument **3** goes to **b**.

SLIDE 4 — Function with Parameters: Flow

Flowchart

None



↓
END

Example

None

```
#include <stdio.h>

void add(int a, int b) {
    printf("%d", a + b);
}

int main() {
    add(5, 3);
    return 0;
}
```

Output

None

8

Explanation

1. **a** receives 5.
2. **b** receives 3.
3. The function adds both values.
4. The result 8 is displayed.

SLIDE 5 — Parameters in Real Life

Witty Explanation

A function is like a college photocopy shop:

- The shop is the function.
- The document is the parameter.

- You give different documents.
- The shop performs the same task each time.

Memory Trick 🧠

Function = Worker Parameter = Work given to the worker Argument = Actual work sent

Questions

1. Theory MCQ What is a parameter used for in a function?

A. To stop the function B. To receive a value C. To delete a variable D. To end the program

💡 Hint: A parameter receives data.

📖 Glossary: Parameter — A variable that receives a value inside a function.

Answer: B. To receive a value

2. Output Guess

None

```
#include <stdio.h>

void add(int a, int b) {
    printf("%d", a + b);
}

int main() {
    add(2, 5);
    return 0;
}
```

What is the output?

💡 Hint: Add the two arguments.

📖 Glossary: Argument — The actual value passed to a function.

Answer:

None

7

3. Fill the Missing Code

None

```
#include <stdio.h>

void display(_____ n) {
    printf("%d", n);
}
```

💡 Hint: The parameter stores a whole number.

📖 Glossary: int — A data type used to store whole numbers.

Answer: `int`

4. Jumbled Sentence

Arrange the words correctly:

function / calling / executes / the code / A

💡 Hint: Start with “Calling”.

📖 Glossary: Calling — Asking a function to run its code.

Answer: Calling a function executes the code.

5. Fill the Missing Code

None

```
#include <stdio.h>

void add(_____ a, _____ b) {
    printf("%d", a + b);
}

int main() {
    add(3, 4);
}
```

```
    return 0;  
}
```

💡 Hint: Both parameters store whole numbers.

📖 Glossary: Multiple parameters — Two or more values received by a function.

Answer:

None

int

int